

Instructions — MapRover Setup Guide

This guide walks through setting up MapRover from scratch. Complete each section in order.

Section 1: Hardware Assembly

1. Assemble the 6WD BO60 chassis following the kit's included assembly manual. Ensure all 6 motors are seated firmly and the rocker-bogie arms pivot freely on both sides.
 2. Attach aluminium heatsinks to the Raspberry Pi 3 before mounting it:
 - Largest heatsink → main processor chip
 - Medium heatsink → USB/Ethernet controller chip
 3. Mount the Raspberry Pi, ESP32, and L298N. Attach the L298N in the center using strong double sided tape. Laser cut or 3D print the plate to mount raspberry pi and use standoffs to make the attachments. Use a breadboard for mounting the ESP32 and use double sided tape to attach the breadboard. Never place circuit boards directly on the metal chassis — it causes short circuits.
 4. Mount the YDLidar X2 at the highest point of the rover, at the geometric center (front-to-back and left-to-right). The laser must have a clear 360° view with no parts of the rover body blocking it. Secure it firmly — vibration corrupts scan data. 3D print the cylinders and the plate for the lidar and use long screws to connect everything.
 5. Attach the MPU-6050 flat (level with the ground) and oriented so the IMU Z-axis points straight up. This is critical for accurate yaw (rotation) measurement. This attachment is to be done on the plate made for the raspberry pi. It has holes for this purpose.
 6. Mount the 11.1V battery low on the chassis for a lower center of gravity. Secure the powerbank on the side or rear.
-

Section 2: Wiring

For the full detailed wiring diagram with pin numbers, see [/Electronics/](#) in the repo root.

Motor wiring:

- Right 2 motors (front-right and rear-right): wire both A terminals together → OUT1, both B terminals together → OUT2
- Left 2 motors (front-left and rear-left): wire both A terminals together → OUT3, both B terminals together → OUT4
- The 2 middle motors on the rocker-bogie arms are left unconnected

ESP32 to L298N connections:

L298N Pin	ESP32 GPIO	Function
ENA	see Code folder	Right side speed (PWM)
IN1	see Code folder	Right direction control
IN2	see Code folder	Right direction control
ENB	see Code folder	Left side speed (PWM)
IN3	see Code folder	Left direction control
IN4	see Code folder	Left direction control

Exact GPIO numbers are in the ESP32 motor control code in [/Code/](#). Check there for the pin assignments specific to your board.

Critical: Connect ESP32 GND to the L298N GND terminal. Without a common ground, the control signals have no reference and the motors will not respond.

Power:

- 11.1V battery (positive) → rocker switch → L298N VCC
- 11.1V battery (negative) → L298N GND
- Powerbank → Raspberry Pi via USB-A to Micro-USB cable (keep completely separate from the motor circuit)

YDLidar X2: Connect to any USB port on the Raspberry Pi using the included USB adapter cable.

MPU-6050 to Raspberry Pi (I2C):

MPU-6050 Pin	Raspberry Pi Physical Pin
VCC	Pin 1 (3.3V)
GND	Pin 6 (GND)
SDA	Pin 3 (GPIO 2)
SCL	Pin 5 (GPIO 3)

Section 3: Raspberry Pi OS and ROS2 Setup

3.1 — Flash Ubuntu Server 22.04 LTS (64-bit) to the microSD card using Raspberry Pi Imager (use a 16GB or larger card). In the imager's OS Customisation settings before writing: set hostname to `mapRover`, enable SSH, set a username and password, and enter your WiFi credentials.

3.2 — SSH into the Pi after first boot:

```
ssh <your-username>@mapRover.local
```

If `mapRover.local` doesn't resolve, find the Pi's IP address from your router's connected devices list and use that instead: `ssh <your-username>@<ip-address>`

3.3 — Install ROS2 Humble:

```
sudo apt update && sudo apt upgrade -y
```

```
sudo apt install -y software-properties-common curl
```

```
sudo curl -sSL https://raw.githubusercontent.com/ros/rosdistro/master/ros.key \
```

```
-o /usr/share/keyrings/ros-archive-keyring.gpg
```

```
echo "deb [arch=$(dpkg --print-architecture) \
```

```
signed-by=/usr/share/keyrings/ros-archive-keyring.gpg] \
```

```
http://packages.ros.org/ros2/ubuntu jammy main" \
```

```
| sudo tee /etc/apt/sources.list.d/ros2.list > /dev/null
```

```
sudo apt update
```

```
sudo apt install -y ros-humble-ros-base python3-colcon-common-extensions
```

```
echo "source /opt/ros/humble/setup.bash" >> ~/.bashrc
```

```
source ~/.bashrc
```

3.4 — Install required packages:

```
sudo apt install -y \
```

```
python3-pip \
```

```
cmake \
```

```
ros-humble-slam-toolbox \
```

```
ros-humble-robot-localization \
```

```
ros-humble-tf-transformations \
```

```
i2c-tools python3-smbus
```

```
pip3 install pyserial smbus2
```

3.5 — Add user to device access groups:

This is required so your user can access the YDLidar (USB serial) and MPU-6050 (I2C) without running commands as root.

```
sudo usermod -aG dialout $USER
```

```
sudo usermod -aG i2c $USER
```

Log out and back in (or reboot) for the group changes to take effect.

3.6 — Enable I2C for the MPU-6050:

```
sudo nano /boot/firmware/config.txt
```

```
# Add at the bottom: dtparam=i2c_arm=on
```

```
echo "i2c-dev" | sudo tee -a /etc/modules
```

```
sudo reboot
```

After reboot, verify the IMU is detected:

```
i2cdetect -y 1
```

You should see address 0x68 in the grid

3.7 — Create the ROS2 workspace:

This step is required before cloning any packages. Without it, steps 3.8 and 3.9 will fail because the directory doesn't exist.

```
mkdir -p ~/ros2_ws/src
```

```
cd ~/ros2_ws
```

```
colcon build
```

```
echo "source ~/ros2_ws/install/setup.bash" >> ~/.bashrc
```

```
source ~/.bashrc
```

3.8 — Install the YDLidar ROS2 driver:

```
cd ~/ros2_ws/src
```

```
git clone https://github.com/YDLIDAR/ydlidar_ros2_driver.git
```

```
cd ~/ros2_ws
```

```
colcon build --symlink-install --packages-select ydlidar_ros2_driver
```

```
source ~/.bashrc
```

After building, install the YDLidar udev rules. These give the device a consistent name (`/dev/ydlidar`) so the launch file always finds it regardless of which USB port it is plugged into:

```
cd ~/ros2_ws/src/ydlidar_ros2_driver
```

```
sudo chmod 0777 startup/initenv.sh
```

```
sudo ./startup/initenv.sh
```

Reconnect the YDLidar USB cable, then verify the device is visible:

```
ls /dev/ydlidar
```

Should print: /dev/ydlidar

3.9 — Clone and build the maprover_core package:

```
cd ~/ros2_ws/src
```

```
git clone https://github.com/MakersAsylumIndia/IS2026_SU_Team16_MapRover.git  
maprover_core
```

```
cd ~/ros2_ws
```

```
colcon build --symlink-install --packages-select maprover_core
```

```
source ~/.bashrc
```

3.10 — Understanding the configuration files:

All config files live inside `~/ros2_ws/src/maprover_core/config/`. You do not need to edit these to run the project as-is, but understanding them is important if you want to adapt the project to different hardware or tune the mapping behaviour.

mapping.launch.py Location: `maprover_core/launch/mapping.launch.py`

This is the single entry point that starts everything. It launches five things in order:

1. The YDLidar driver node (with the lidar params file)
2. The IMU publisher node (reads MPU-6050 and publishes to `/imu/data`)
3. The static transform publishers (defines the physical positions of sensors on the rover)
4. The robot_localization EKF node (with the EKF params file)

5. The slam_toolbox async mapping node (with the slam params file)

If any of these fail to start, run `ros2 launch maprover_core mapping.launch.py --debug` to see which node is throwing errors.

ydlidar_params.yaml Location: `maprover_core/config/ydlidar_params.yaml`

Tells the YDLidar ROS2 driver how to communicate with the X2 sensor.

Key parameters and what to change:

port: /dev/ydlidar # Device name — only change if udev rules weren't installed

baudrate: 115200 # Fixed for X2, do not change

lidar_type: 1 # 1 = TYPE_TRIANGLE, correct for X2

frame_id: laser_frame # Must match the TF static transform (see below)

angle_min: -3.14159 # Full 360° scan, $-\pi$ to π

angle_max: 3.14159

range_min: 0.10 # Minimum valid scan distance in metres

range_max: 8.0 # X2 maximum range is 8m

If the lidar appears to scan only half the room or produces a mirrored map, check `angle_min/angle_max` and the physical mounting orientation.

ekf_params.yaml Location: `maprover_core/config/ekf_params.yaml`

Configures the `robot_localization` Extended Kalman Filter. The EKF fuses the IMU's gyroscope data to produce a stable `odom` → `base_link` transform for slam_toolbox to use. Without this, sharp turns cause the map to tear because slam_toolbox has no reliable rotation reference between scans.

Key parameters and what to change:

```

frequency: 30          # How often the EKF publishes, in Hz

imu0: /imu/data        # Topic published by the IMU node — must match exactly

imu0_config: [false, false, false, # x, y, z position    — not used

              false, false, true,  # roll, pitch, yaw    — yaw (rotation) is used

              false, false, false, # vx, vy, vz velocity — not used

              false, false, true,  # vroll, vpitch, vyaw  — yaw rate is used

              false, false, false] # ax, ay, az accel    — not used

odom_frame: odom

base_link_frame: base_link

world_frame: odom

```

The `imu0_config` matrix is the most important part. Each `true` entry tells the EKF to use that measurement from the IMU. For this project only yaw (rotation around Z axis) and yaw rate matter — position and linear velocity from the IMU are too noisy to be useful.

If the map drifts or rotates incorrectly, the first thing to check is whether `/imu/data` is being published (`ros2 topic echo /imu/data`) and whether the covariance values in the IMU message are set correctly by the IMU node.

slam_toolbox_params.yaml Location:
`maprover_core/config/slam_toolbox_params.yaml`

Configures the `slam_toolbox` async mapping node. This controls map resolution, how often the map updates, and how far the rover must move before a new scan is processed.

Key parameters and what to change:

```

mode: mapping          # 'mapping' builds a new map; 'localization' uses a saved one

resolution: 0.05       # Map resolution in metres per pixel (0.05 = 5cm per cell)

                      # Lower = more detailed map but slower on Pi 3

```


max_laser_range: 8.0 # Should match the lidar's range_max in ydlidar_params.yaml

minimum_travel_distance: 0.3 # Rover must move 0.3m before processing a new scan

minimum_travel_heading: 0.3 # Or rotate 0.3 radians (~17°)

 # Increase these if the Pi is struggling to keep up

map_update_interval: 1.0 # Seconds between map publishes to RViz2

 # Increase if map updates are causing lag

odom_frame: odom

base_frame: base_link

map_frame: map

scan_topic: /scan # Must match what the YDLidar driver publishes

If the map looks blocky, lower the **resolution**. If the Pi is running hot or the map lags, increase **minimum_travel_distance** and **map_update_interval**.

imu_publisher.py Location: `maprover_core/maprover_core/imu_publisher.py`

This is a custom ROS2 node because the MPU-6050 has no official ROS2 driver. It reads raw gyroscope and accelerometer data from the sensor over I2C using **smbus2**, converts it to standard SI units, and publishes a **sensor_msgs/Imu** message to **/imu/data** at 30Hz.

What it does:

- Reads gyroscope registers (rotation rate in radians/second around X, Y, Z)
- Reads accelerometer registers (linear acceleration in m/s²)
- Fills in the **orientation_covariance**, **angular_velocity_covariance**, and **linear_acceleration_covariance** fields — these tell the EKF how much to trust the sensor
- Publishes with **frame_id: imu_link**

If the IMU node is running but the map still tears on turns, the covariance values in this file may need tuning — lower covariance values tell the EKF to trust the IMU more.

Static Transform Publishers (TF tree)

The TF (Transform) tree defines the physical geometry of the rover — where each sensor is mounted relative to the robot's center. `slam_toolbox` uses this to know where the laser scan originated in space. Without a correct TF tree, the mapping will fail silently or produce a distorted map.

The full TF chain for this project is:

`map → odom → base_link → laser_frame`

`→ imu_link`

- `map → odom` is published live by `slam_toolbox`
- `odom → base_link` is published live by the EKF node
- `base_link → laser_frame` is a static transform defined in the launch file
- `base_link → imu_link` is a static transform defined in the launch file

The static transforms look like this in the launch file:

Node(

`package='tf2_ros',`

`executable='static_transform_publisher',`

`arguments=['0', '0', '0.1', '0', '0', '0', 'base_link', 'laser_frame']`

`# x=0, y=0, z=0.1m above base, no rotation`

)

If the LiDAR is remounted at a different height or position on the rover, update the `z` value (height in metres) in this static transform. Getting this wrong does not crash anything but subtly corrupts the map geometry.

To verify the full TF tree is correctly connected at runtime:

`ros2 run tf2_tools view_frames`

This generates a `frames.pdf` showing the complete tree. Every frame must be connected — any gap in the chain means mapping will not work.

Section 4: ESP32 Setup

4.1 — Install Arduino IDE on your computer from <https://www.arduino.cc/en/software>

4.2 — Add ESP32 board support:

- File → Preferences → Additional Board Manager URLs:
`https://espressif.github.io/arduino-esp32/package_esp32_index.json`
- Tools → Board → Boards Manager → search "esp32" → install "esp32 by Espressif Systems" (version 3.0 or newer)

4.3 — Install required libraries:

- Sketch → Include Library → Manage Libraries
- Install: `ESPAsyncWebServer` and `AsyncTCP`

4.4 — Flash the ESP32 motor control code:

- Open `/Code/ESP32_motor_control/` from this repo in Arduino IDE
 - Select board: Tools → Board → esp32 → "ESP32 Dev Module"
 - Select the correct COM port under Tools → Port
 - Upload the code
-

Section 5: Laptop Setup (RViz2 via WSL2 on Windows)

5.1 — Install WSL2 on Windows (if not already installed):

```
wsl --install
```

Restart your computer. Open the Ubuntu app from the Start menu and complete the initial username/password setup.

5.2 — Check your Windows version for GUI support:

RViz2 is a graphical application. How it works depends on your Windows version:

- **Windows 11:** WSLg is built in and handles GUI apps automatically. No extra setup needed.
- **Windows 10:** WSL2 does not include GUI support. You must install **VcXsrv** (free X server) from <https://sourceforge.net/projects/vcxsrv/>. After installing, launch XLaunch, choose "Multiple Windows", set Display number to 0, and tick "Disable access control". Then in WSL2 run:

```
echo "export DISPLAY=$(cat /etc/resolv.conf | grep nameserver | awk '{print $2}'):0" >>
~/.bashrc
```

```
source ~/.bashrc
```

5.3 — Install ROS2 Humble in WSL2 following the same steps as Section 3.3 above.

5.4 — Install RViz2:

```
sudo apt install -y ros-humble-rviz2
```

5.5 — Create the ROS2 workspace in WSL2:

```
mkdir -p ~/ros2_ws/src
```

```
cd ~/ros2_ws
```

```
colcon build
```

```
echo "source ~/ros2_ws/install/setup.bash" >> ~/.bashrc
```

```
source ~/.bashrc
```

5.6 — Fix WSL2 networking for ROS2 cross-machine discovery:

By default, WSL2 uses a virtual NAT network which blocks the multicast packets that ROS2 uses to discover nodes on other machines. Without this fix, the laptop will never see topics from the Raspberry Pi even if both are on the same WiFi.

On Windows 11: Enable mirrored networking mode. Open (or create) `C:\Users\<YourUsername>\.wslconfig` in Notepad and add:

```
[wsl2]
```

networkingMode=mirrored

Then restart WSL2 from PowerShell: `wsl --shutdown`, then reopen Ubuntu.

On Windows 10: Mirrored mode is not available. Use CycloneDDS with a manual peer config. Install it in WSL2:

```
sudo apt install -y ros-humble-rmw-cyclonedds-cpp
```

```
echo "export RMW_IMPLEMENTATION=rmw_cyclonedds_cpp" >> ~/.bashrc
```

Create a CycloneDDS config file:

```
nano ~/cyclonedds_pi.xml
```

Add the following, replacing `<PI_IP_ADDRESS>` with your Raspberry Pi's actual IP:

```
<CycloneDDS>
```

```
<Domain>
```

```
<Discovery>
```

```
<Peers>
```

```
<Peer Address="<PI_IP_ADDRESS>"/>
```

```
</Peers>
```

```
</Discovery>
```

```
</Domain>
```

```
</CycloneDDS>
```

Then add to `.bashrc`:

```
echo "export CYCLONEDDS_URI=file://$HOME/cyclonedds_pi.xml" >> ~/.bashrc
```

```
source ~/.bashrc
```

Apply the same CycloneDDS setup on the Raspberry Pi as well.

5.7 — Set the ROS domain ID (must match the Pi):

```
echo "export ROS_DOMAIN_ID=42" >> ~/.bashrc
```

```
echo "export ROS_LOCALHOST_ONLY=0" >> ~/.bashrc
```

```
source ~/.bashrc
```

On the Raspberry Pi, set the same:

```
echo "export ROS_DOMAIN_ID=42" >> ~/.bashrc
```

```
echo "export ROS_LOCALHOST_ONLY=0" >> ~/.bashrc
```

```
source ~/.bashrc
```

Both machines must be on the same WiFi network. After this, they will automatically discover each other's ROS2 topics. Verify it is working by running `ros2 topic list` in WSL2 while the Pi's mapping stack is running — you should see `/map`, `/scan`, and `/imu/data` in the list.

Section 6: Running the Rover

Step 1 — Start mapping on the Raspberry Pi:

```
ros2 launch maprover_core mapping.launch.py
```

This launches the YDLidar driver, the IMU publisher, the EKF node, and `slam_toolbox` together.

Step 2 — Open RViz2 on your laptop (WSL2):

```
export DISPLAY=:0
```

```
rviz2
```

In RViz2:

- Set Fixed Frame to `map`
- Add → Map → Topic: `/map`
- Add → LaserScan → Topic: `/scan`

Step 3 — Connect to the rover's WiFi:

- Connect your phone or laptop to the ESP32's WiFi hotspot (network name and password are in the ESP32 code)
- Open a browser and go to `192.168.4.1`
- Drive the rover — the map builds live as you move

Optional — Xbox controller: Connect the controller via Bluetooth or USB to the device running the browser. The web interface supports Xbox controller input via the browser Gamepad API.

Saving the map: Once you have mapped an area, save it with:

```
ros2 service call /slam_toolbox/save_map slam_toolbox/srv/SaveMap "{name: {data: '/home/<username>/my_map'}}"
```

This creates `my_map.pgm` and `my_map.yaml`.

Known Issues and Notes

- The Raspberry Pi 3 is resource-limited for SLAM. Expect the map update rate to be around 1 second. If the system becomes sluggish, ensure no unnecessary processes are running.
- RViz2 on WSL2 may require `export DISPLAY=:0` or `export DISPLAY=:0.0` before launching, depending on your WSL2 version.
- The YDLidar X2 must be connected before the launch file is run, or the driver node will fail to start.
- If the map tears diagonally during sharp turns, check that the IMU publisher and EKF nodes are running correctly (`ros2 topic list` should show `/imu/data` and an `odom` → `base_link` transform should exist in the TF tree).
- If the motor driver shows abnormal behaviour (such as one side not rotating in one of the directions) make sure to change your jumper wires before anything else.
- With the middle two wheels not turning, resistance increases so for higher speed movement try using two motor drivers. This however would not help from the mapping perspective as the quality of the map is better the slower the rover moves.